Introduction to Generative AI and Large Language Models

Introduction

This lesson will cover:

- Introduction to the business scenario: our startup idea and mission.
- Generative AI and how we landed on the current technology landscape.
- Inner working of a large language model.
- Main capabilities and practical use cases of Large Language Models.

Learning Goals

After completing this lesson, you will understand:

- What generative AI is and how Large Language Models work.
- How you can leverage large language models for different use cases, with a focus on education scenarios.

Scenario: our educational startup

Generative Artificial Intelligence (AI) represents the pinnacle of AI technology, pushing the boundaries of what was once thought impossible. Generative AI models have several capabilities and applications, but for this curriculum we'll explore how it's revolutionizing education through a fictional startup. We'll refer to this startup as *our startup*. Our startup works in the education domain with the ambitious mission statement of

> *improving accessibility in learning, on a global scale, ensuring equitable access to education and providing personalized learning experiences to every learner, according to their needs*.

How did we get Generative AI?

Despite the extraordinary *hype* created lately by the announcement of generative AI models, this technology is decades in the making, with the first research efforts dating back to the 60s. We're now at a point with AI having human cognitive capabilities, like conversation as shown by for example OpenAI ChatGPT or Microsoft Copilot, which also uses a GPT model for its conversational web search experience.

Backing up a bit, the very first prototypes of AI consisted of typewritten chatbots, relying on a knowledge base extracted from a group of experts and represented into a computer. The answers in the knowledge base were triggered by keywords appearing in the input text. However, it soon became clear that such an approach, using typewritten chatbots, did not scale well.

A statistical approach to AI: Machine Learning

A turning point arrived during the 90s, with the application of a statistical approach to text analysis. This led to the development of new algorithms – known as machine learning – capable of learning patterns from data without being explicitly programmed. This approach allows machines to simulate human language understanding: a statistical model is trained on text-label pairings, enabling the model to classify unknown input text with a pre-defined label representing the intention of the message.

Neural networks and modern virtual assistants

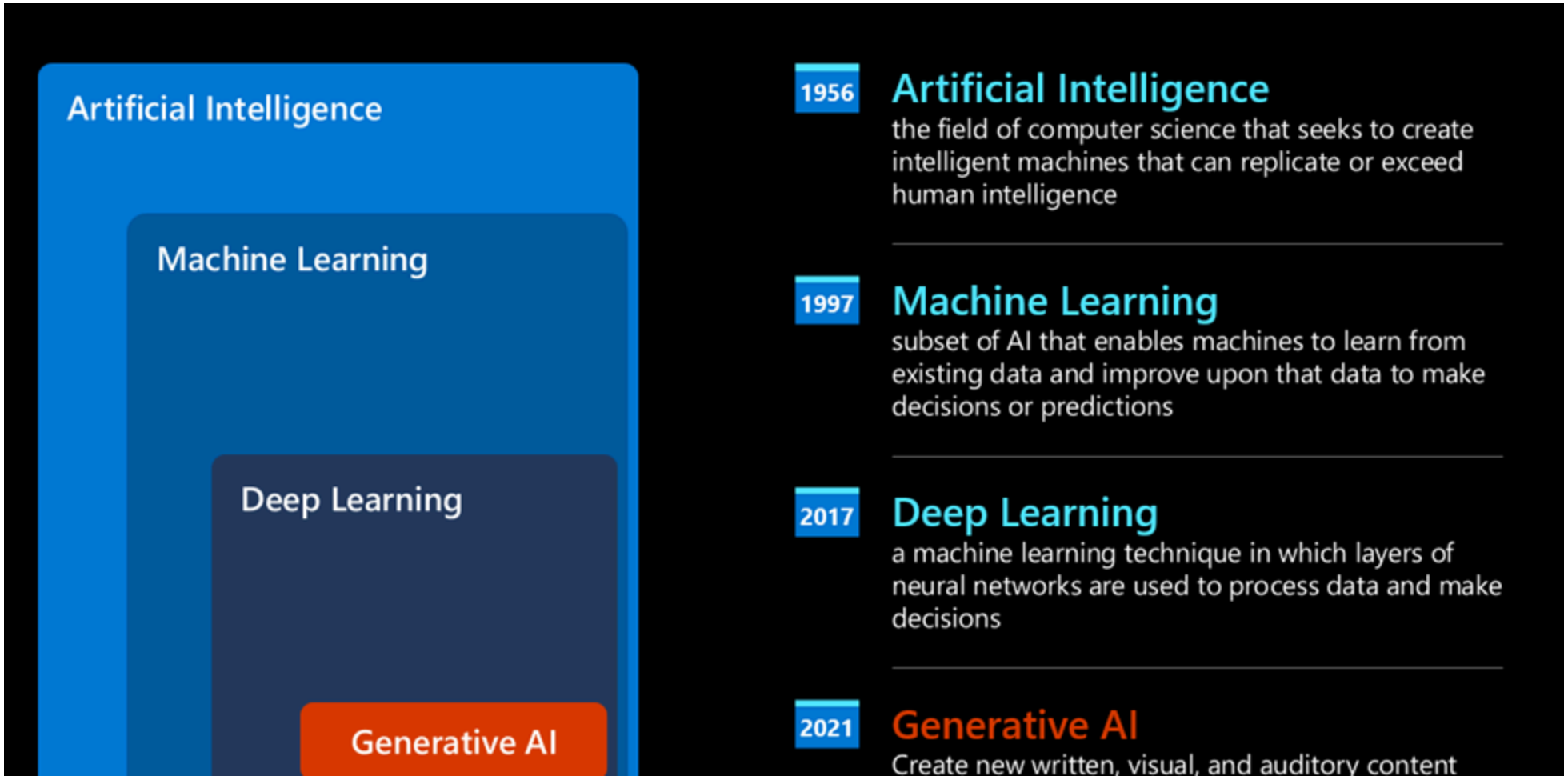
In recent years, the technological evolution of hardware, capable of handling larger amounts of data and more complex computations, encouraged research in AI, leading to the development of advanced machine learning algorithms known as neural networks or deep learning algorithms.

Neural networks (and in particular Recurrent Neural Networks – RNNs) significantly enhanced natural language processing, enabling the representation of the meaning of text in a more meaningful way, valuing the context of a word in a sentence.

This is the technology that powered the virtual assistants born in the first decade of the new century, very proficient in interpreting human language, identifying a need, and performing an action to satisfy it – like answering with a pre-defined script or consuming a 3rd party service.

Present day, Generative AI

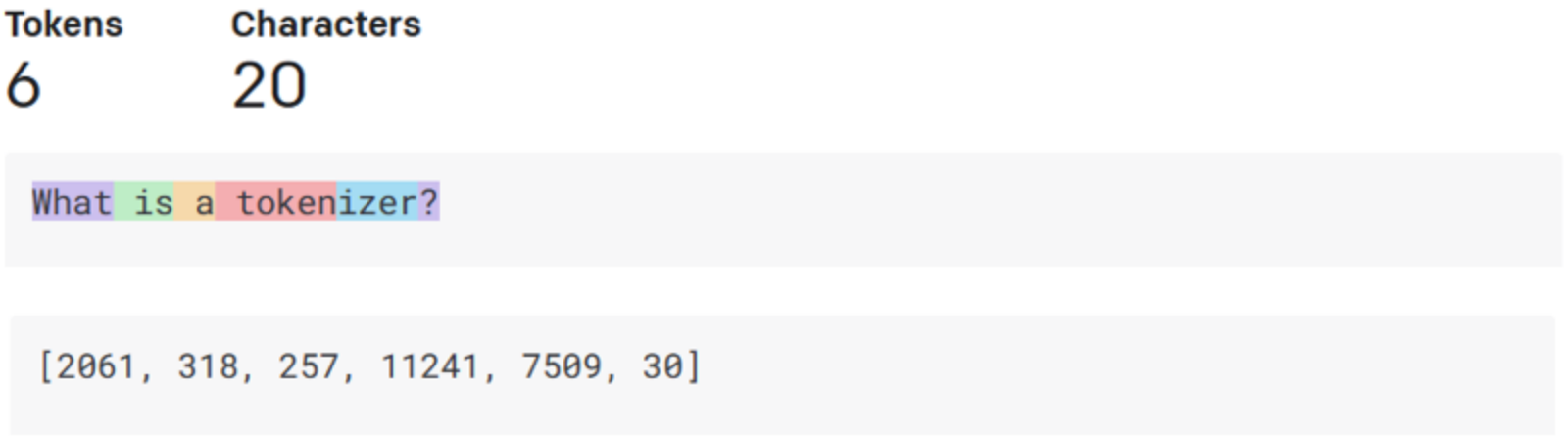
So that's how we came to Generative AI today, which can be seen as a subset of deep learning.



How do large language models work?

In the next chapter we are going to explore different types of Generative AI models, but for now let’s have a look at how large language models work, with a focus on OpenAI GPT (Generative Pre-trained Transformer) models.

- **Tokenizer, text to numbers**: Large Language Models receive a text as input and generate a text as output. However, being statistical models, they work much better with numbers than text sequences. That’s why every input to the model is processed by a tokenizer, before being used by the core model. A token is a chunk of text – consisting of a variable number of characters, so the tokenizer's main task is splitting the input into an array of tokens. Then, each token is mapped with a token index, which is the integer encoding of the original text chunk.



- **Predicting output tokens**: Given n tokens as input (with max n varying from one model to another), the model is able to predict one token as output. This token is then incorporated into the input of the next iteration, in an expanding window pattern, enabling a better user experience of getting one (or multiple) sentence as an answer. This explains why, if you ever played with ChatGPT, you might have noticed that sometimes it looks like it stops in the middle of a sentence.

- **Selection process, probability distribution**: The output token is chosen by the model according to its probability of occurring after the current text sequence. This is because the model predicts a probability distribution over all possible ‘next tokens’, calculated based on its training. However, not always is the token with the highest probability chosen from the resulting distribution. A degree of randomness is added to this choice, in a way that the model acts in a non-deterministic fashion - we do not get the exact same output for the same input. This degree of randomness is added to simulate the process of creative thinking and it can be tuned using a model parameter called temperature.

How can our startup leverage Large Language Models?

- An **instruction** specifying the type of output we expect from the model. This instruction sometimes might embed some examples or some additional data.
    i. Summarization of an article, book, product reviews and more, along with extraction of insights from unstructured data.
    ii. Creative ideation and design of an article, an essay, an assignment or more.
- A **question**, asked in the form of a conversation with an agent.
- A chunk of **text to complete**, which implicitly is an ask for writing assistance.
- A chunk of **code** together with the ask of explaining and documenting it, or a comment asking to generate a piece of code performing a specific task.